



Universidad
Tecmilenio

UNIVERSIDAD TECMILENIO
CAMPUS LAS TORRES

Proyecto Final

MATERIA: Ingeniería en desarrollo de software

MAESTRO: Ing. Ricardo Arturo Elizalde Hernández

ALUMNO: Diego Villarreal Martínez

MATRICULA: AI07064821

CARRERA: Ingeniería de Desarrollo de Software

Link de GitHub:

https://github.com/WebbiestStew/finalproj_ids.git

Link de la pagina:

<https://dauto-ym8c.onrender.com>

1. Implementación y seguridad

1.1 Módulo: autenticación de concesionarias y compradores

Del módulo **Auth & Usuarios** de la arquitectura (sección 2.3) se implementó el flujo completo, de la API a la interfaz

Backend (**backend/**)

- JWT firmado con **HS256** y **algoritmo fijado al verificar** (un token **HS512** o **alg: none** se rechaza), 2 h de vigencia. Contraseñas con bcrypt asíncrono (factor 12; configurable con **BCRYPT_ROUNDS**, mínimo 10 en producción según OWASP)
- Roles **admin**, **concesionaria**, **comprador**. El registro público **no** permite auto-asignarse **admin**.
- Endpoints: **POST /api/auth/register**, **POST /api/auth/login**, **GET /api/auth/me**, **GET /api/auth/users** (solo admin), **GET /health**
- Validación con **zod**, consultas parametrizadas, **helmet**, límite de intentos, cuerpo máximo de 10 kB.

Frontend (frontend/**)** — SPA en React: landing, registro (con selector de rol y validación en línea), inicio de sesión, y un panel que cambia por rol (el administrador ve la tabla de usuarios). Rutas protegidas, rutas solo para invitados, regreso a la página que se intentaba abrir tras iniciar sesión y cierre de sesión automático si el token vence

Catálogo de vehículos (adelanto del Sprint 2) — API y pantallas para publicar y consultar autos, la funcionalidad central del acta (secciones 1.1 y 2.1, historia de usuario 1 y 2):

- **API:** **GET /api/vehicles** (público, con filtros por marca/modelo, carrocería, transmisión, precio, año y kilometraje; orden; paginación), detalle, marcas, y para concesionarias **POST/PUT/PATCH/DELETE** con **control de propiedad**: cada concesionaria solo modifica sus autos y el admin puede moderar. Un auto **vendido** deja de aparecer en el catálogo público, lo que mitiga el riesgo **R4** de la matriz (inventario desactualizado). El orden se elige de una lista fija (nunca se concatena entrada del usuario al SQL) y los comodines **%/_** de la búsqueda se escapan
- **Frontend:** catálogo con filtros sincronizados con la URL (enlaces compartibles), "Ver más", ficha de detalle, y un módulo de **inventario** para concesionarias (publicar, editar con vista previa, cambiar estado, eliminar con confirmación). La landing muestra los autos más recientes
- **Alcance honesto:** todavía no hay carga de fotos; cada auto se muestra con una **ilustración** cuya silueta y color salen de sus datos, y la ficha lo indica. La geolocalización de sucursales, el financiamiento y las citas siguen pendientes

Pruebas unitarias y de integración

	Pruebas	Statements	Branches	Functions	Lines	Umbral
Backend (Jest + supertest)	130	100 %	93.89 %	100 %	100 %	80 % (rompe la build)
Frontend (Vitest + Testing Library)	143	97.36 %	87.64 %	96.50 %	97.83 %	—

Las pruebas del backend cubren registro/login por rol, roles, middleware, y las pruebas de seguridad negativa (SQLi, XSS almacenado, tokens con otro algoritmo/sin firma/expirados/con otro secreto, JSON mal formado, cuerpo demasiado grande, límite de intentos, cabeceras). Las del frontend cubren el contexto de autenticación (incluye que un fallo de red **no** cierra la sesión), guardas de ruta, formularios, accesibilidad (**role="alert"**, **aria-invalid**, **aria-describedby**, **aria-pressed**, teclado) y el panel por rol

1.2 Pipeline CI/CD (GitHub Actions)

`.github/workflows/ci-cd.yml`:

1. **lint-and-test** (backend): `npm audit` de producción → ESLint + sonarjs → duplicación (`jscpd`, falla sobre 3 %) → pruebas con cobertura (falla bajo 80 %)
2. **frontend**: `npm audit` → `oxlint` → pruebas con cobertura → build de producción
3. **build-and-push** (solo `main`, requiere los dos anteriores): imagen Docker (usuario no root) a GHCR
4. **deploy-test-env**: ejecuta la imagen publicada en un contenedor efímero y valida `/health`, el frontend, el catálogo y un registro y login reales

`.github/workflows/security-scan.yml` corre OWASP ZAP contra el servidor levantado y SonarCloud (con el secreto `SONAR_TOKEN`); este último job además exporta métricas, *quality gate* y hallazgos como reporte

5. **deploy-hosting**: si existe el secreto `RENDER_DEPLOY_HOOK`, dispara el despliegue en Render tras pasar todo lo anterior; sin el secreto se omite y el pipeline queda en verde

Una sola imagen, un solo servicio. El backend sirve el frontend compilado en el mismo puerto (con caché inmutable para `assets/`, `index.html` siempre revalidado y rutas del cliente que sobreviven a un refresco), por lo que desplegar es publicar una imagen: no hay CORS ni segundo servicio. Un solo comando en local (`npm run setup && npm run dev`) y otro con Docker (`docker compose up --build`); `render.yaml` y `fly.toml` dejan listos dos hostings, y `docs/despliegue.md` explica todo paso a paso. El administrador y los datos demo se crean desde variables de entorno, lo que permite arrancar en un hosting sin terminal

2. Pruebas y calidad

2.1 Pruebas de seguridad

Vías complementarias: OWASP ZAP baseline en CI, pruebas manuales dirigidas y revisión de código.

#	Hallazgo	Origen	Corrección	Verificación
1	XSS almacenado: <code><script></code> se guardaba tal cual en <code>name</code>	Prueba manual	Regla en <code>zod</code> que rechaza <code>< y ></code> (y la misma regla en el frontend)	Prueba de regresión
2	<code>Access-Control-Allow-Origin: *</code> (ZAP: <i>Cross-Domain Misconfiguration</i>)	ZAP, corrida real de CI	Lista blanca de orígenes; nunca <code>*</code> ; en producción sin configuración no se permite ningún origen	<code>hardening.test.js</code>
3	Respuestas almacenables en caché (ZAP: <i>Storable and Cacheable Content</i>)	ZAP, corrida real de CI	<code>Cache-Control: no-store</code> global (incluye 404) y ETag desactivado	<code>hardening.test.js</code>
4	Algoritmo del JWT sin fijar (riesgo de confusión de algoritmo / <code>alg: none</code>)	Revisión de código	<code>algorithms: ['HS256']</code> al verificar y al firmar	Pruebas con HS512, <code>none</code> , expirado y otro secreto
5	Enumeración de cuentas por tiempo de respuesta (correo inexistente respondía más rápido)	Revisión de código	Se compara contra un hash señuelo cuando el correo no existe	Prueba de respuesta idéntica
6	JSON mal formado devolvía 500	Prueba manual	400 (y 413 para cuerpos grandes); el 500 nunca expone detalles	<code>hardening.test.js</code>
7	<code>bcrypt</code> síncrono bloqueaba el event loop	Revisión de código	<code>bcrypt.hash/compare</code> asíncronos + <code>asyncHandler</code>	Suite completa
8	SQLi en <code>email</code> (registro/login)	Prueba manual	Ya seguro por diseño (<code>zod</code> + consultas parametrizadas)	Prueba de regresión
9	Cabeceras de seguridad	Prueba manual + ZAP	<code>helmet</code> correcto	Confirmado
10	Referencia directa insegura (IDOR): una concesionaria podría editar, cambiar de estado o borrar autos de otra cambiando el <code>:id</code>	Diseño del catálogo	Se carga el vehículo y se compara su dueña con el token en cada operación de escritura; el admin puede moderar	Pruebas de 403 en PUT, PATCH y DELETE y de que el auto queda intacto
12	Escrituras sin límite: una cuenta podía publicar sin tope y a cualquier frecuencia (inflar la base y el catálogo)	Revisión de diseño	Máximo de 500 vehículos por concesionaria (409) y límite de frecuencia sobre POST/PUT/PATCH/DELETE del catálogo (429)	<code>deploy.test.js</code>
13	Datos demo con contraseña conocida (<code>demo-password-123</code>) habrían llegado a un despliegue público	Revisión de despliegue	Sin <code>DEMO_PASSWORD</code> las cuentas demo se crean con una contraseña aleatoria imposible de adivinar; el pipeline verifica que un login con la contraseña conocida devuelve 401	<code>deploy.test.js</code> y prueba de humo de CI
11	Comodines <code>%/_</code> en la búsqueda del catálogo y <code>ORDER BY</code> con entrada del usuario	Diseño del catálogo	Se escapan los comodines; el orden solo elige entre fragmentos SQL fijos	Pruebas de búsqueda con <code>%</code> y con inyección SQL, y de orden inválido → 400

Estado de ZAP: la corrida baseline previa a las correcciones reportó 65 reglas superadas y 2 advertencias (las de los hallazgos 2 y 3); se abrió automáticamente el issue de GitHub *"ZAP Scan Baseline Report"*. Tras las correcciones, una advertencia adicional (*Non-Storable Content*) era la consecuencia esperada de **Cache-Control: no-store** y se registró como decisión de diseño en **.zap/rules.tsv**. La corrida final reporta **0 fallos, 0 advertencias, 66 reglas superadas y 1 ignorada**, y el issue se cerró solo (*"All the alerts have been resolved"*).
Reporte: **docs/reportes/zap-baseline-final.txt**.

Riesgo aceptado y documentado: el JWT se guarda en **localStorage** (legible por cualquier script de la página). Es el patrón habitual de una SPA con API sin estado; se mitiga con la validación anti-XSS de entrada y el escapado por defecto de React, y su migración a cookie **httpOnly** (con protección CSRF) está en el plan de mejora.

2.2 Calidad de código y métricas

El análisis oficial se hace con **SonarCloud** (la versión en la nube de SonarQube) sobre backend y frontend (4 616 líneas de código), integrado al workflow `security-scan.yml`. El job exporta las métricas, el resultado del *quality gate* y los hallazgos abiertos a `docs/reportes/sonarcloud/` y como artefacto de cada corrida.

Métrica de SonarCloud	Primer análisis	Tras corregir
Quality gate (<i>Sonar way</i>)	Sin calcular (primera corrida)	Aprobado (OK)
Bugs	0	0
Vulnerabilidades	0	0
Security hotspots	0	0
Code smells	2	0
Deuda técnica (<i>sqale_index</i>)	25 min	0 min
Ratings de seguridad, fiabilidad y mantenibilidad	A / A / A	A / A / A
Cobertura	93.6 %	93.6 % (código nuevo: 100 %)
Duplicación	0.0 %	0.0 %

Los dos *code smells* del primer análisis eran reales y se corrigieron:

- 58786 en `validateRegistration.js`: la expresión regular del correo (`^\S+@\S+\.\S+$`) tiene retroceso super-lineal y era un riesgo menor de denegación de servicio por expresiones regulares (ReDoS). Se substituyó por una comprobación lineal (`isValidEmail`) con pruebas, incluida una con entrada hostil de 40 000 caracteres.
- 56852 en `Register.jsx`: el grupo de selección de rol (`role="radiogroup"`) no era enfocable; se añadió `tabIndex=-1` sin cambiar la navegación por teclado.

Además del análisis oficial, estas herramientas locales corren en cada push:

Métrica	Herramienta	Resultado
Code smells / bugs (backend)	ESLint + <code>eslint-plugin-sonarjs</code>	0 (hubo 3 al inicio, ya corregidos)
Warnings (frontend)	oxlint	0 (hubo 3 al inicio: efectos con <code>setState</code> y exportaciones mixtas)
Duplicación de código	<code>jscpd</code>	0.53 % de líneas (1 clon de 10 líneas); umbral en CI: 3 %
Vulnerabilidades de dependencias	<code>npm audit</code>	0 en backend y frontend. Durante la entrega se detectó y evitó una dependencia de desarrollo vulnerable (<code>@vitest/mocker</code> , versiones $\leq 4.1.10$) fijando <code>vitest ^4.1.11</code>
Cobertura	Jest / Vitest	ver 4.1 (frontend: 143 pruebas)
Accesibilidad de color (WCAG AA)	Script de contraste propio	Todos los pares de texto $\geq 4.5:1$ en claro y oscuro, incluidos los <i>badges</i> teñidos sobre la superficie más oscura (peor caso). Una primera versión tenía 5 combinaciones entre 4.15 y 4.49; se corrigieron con tokens de texto más oscuros

2.3 Rendimiento (requisito no funcional de la sección 2.1)

El acta exige **1 000 usuarios concurrentes con respuestas menores a 2 s**. Se midió con **autocannon** sobre una instancia en una laptop de 8 núcleos:

Prueba	Resultado
GET /api/auth/me (verificar JWT + leer SQLite), 100 conexiones	~11 000 req/s, p99 = 11 ms, 0 errores
GET /api/auth/me, 250 y 500 conexiones simultáneas nuevas	Aparecen errores de conexión y p99 de varios segundos
POST /api/auth/login, bcrypt costo 12, 10 concurrentes	3.75 logins/s ; ~2.4 s por login
POST /api/auth/login, bcrypt costo 10	16.75 logins/s (~4.5×)

Conclusiones honestas:

- **El requisito no queda demostrado en este entorno.** Con ≥ 250 conexiones nuevas a la vez, la cola de aceptación TCP de macOS (`kern.ipc.somaxconn = 128`) descarta conexiones antes de que Node las vea, y eso infla la cola de latencias; es una limitación de la máquina de prueba, no medición de la aplicación. Debe repetirse en Linux (CI o staging) con una rampa progresiva.
- **El cuello de botella real es el hash de contraseñas**, no la API: a costo 12 una sola instancia atiende ~4 logins por segundo. "1 000 usuarios concurrentes" con inicio de sesión simultáneo no se sostendría; con tráfico autenticado normal (cientos de req/s) hay más de 10× de margen. Por eso el factor de trabajo es configurable (con piso de 10 en producción) y la mejora de fondo está en el plan (bcrypt nativo o Argon2 en hilos, y escalado horizontal).

3. Cierre y evaluación

3.1 Línea de tiempo planificada vs. real

Hito	Planificado (Gantt, sección 3.2)	Real	Diferencia
Sprint 1 — UI/UX y prototipado	14 → 28-sep-2026	Interfaz implementada y funcional (landing, registro, login, panel) el 18-sep-2026	Dentro de su propia ventana, ~10 días antes de que cierre
Sprint 2 — módulo de autenticación	28-sep → 19-oct-2026 (con catálogo)	Autenticación completa el 17-sep-2026	11 días antes de su inicio planificado
Sprint 6 — QA y seguridad	30-nov → 10-dic-2026	Pruebas de seguridad, cobertura, análisis de calidad y prueba de carga ejecutadas el 17 y 18-sep	Adelantadas ~2.5 meses (<i>shift-left</i>); en Sprint 6 solo queda repetirlas sobre más módulos
Pipeline CI/CD en verde	Implícito desde el primer push	Primer push el 17-sep con ambos workflows en rojo ; en verde el mismo día tras tres correcciones	+0 días de calendario, pero reveló que "escribir" y "tener funcionando" un pipeline son hitos distintos
Catálogo de vehículos (resto de Sprint 2)	28-sep → 19-oct	Publicar, consultar, filtrar y administrar autos: 18-sep-2026 (sin fotos, sin geolocalización)	10 días antes de su ventana; quedan fotos reales y mapa para el resto del sprint

3.2 Planificado vs. ejecutado (alcance)

Punto	Planificado	Ejecutado	Desviación
Módulo + JWT/roles	Auth con roles admin/usuario	3 roles reales del dominio, más un frontend completo	Ampliación: el módulo se validó de punta a punta, no solo por API
Cobertura ≥ 80 %	Backend	Backend 100 % líneas; además frontend con 143 pruebas y 97.83 % de líneas	Por encima de lo pedido
CI/CD con despliegue a entorno de prueba	Verde desde el primer push, entorno persistente	Verde tras 3 correcciones; entorno efímero; ahora cubre también el frontend	Doble desviación real (sin hosting de prueba; primer intento en rojo)
Escaneo OWASP ZAP	XSS/SQLi	ZAP baseline en CI + pruebas manuales; dos advertencias reales corregidas	Se dividió en dos vías por falta de Docker local
SonarQube	Deuda técnica y code smells	SonarCloud activado en CI: 2 code smells y 25 min de deuda en el primer análisis, 0 y 0 min tras corregirlos; <i>quality gate</i> aprobado	Se usó SonarCloud (misma tecnología, en la nube) en lugar de un servidor propio; la activación llegó el último día
Requisito de 1 000 usuarios concurrentes	(sección 2.1)	Medido; no demostrado , cuello de botella identificado	Pendiente de validar en Linux
Catálogo de vehículos	No pedido en el checklist de esta entrega	API con propiedad por concesionaria + catálogo + inventario, con pruebas	Ampliación: primer entregable funcional de negocio, no solo autenticación
Diseño de interfaz	No especificado en el checklist	Rediseño completo con lenguaje de diseño de Apple, WCAG AA medido, modos claro/oscuro y preferencias de accesibilidad	Ampliación de alcance

3.3 Lecciones aprendidas

1. **Validar el formato no basta.** SQLi se bloqueó gratis con consultas parametrizadas y `zod`, pero `<script>` en el nombre pasaba: cada campo libre necesita una regla pensada para el contexto donde se va a renderizar.
2. **Las herramientas asumen infraestructura que no siempre existe** (Docker para ZAP, un servidor para SonarQube). Definir desde el Sprint 0 qué se valida en local y qué solo en CI evita replanificar a la mitad. Aquí la solución fue SonarCloud, pero se activó tarde: el primer análisis real encontró 2 problemas que las herramientas locales equivalentes no habían visto (una regex con retroceso y un fallo de accesibilidad), y es un argumento para activarlo desde el primer sprint.
3. **Un umbral en la configuración (cobertura, duplicación) es una regla; una buena práctica revisada a mano es un deseo.**
4. **"Escribir" un pipeline no es "tener" un pipeline.** El primer push dejó todo en rojo por tres causas que no se ven leyendo el YAML: (a) Docker exige nombres de imagen en minúsculas y el repositorio tiene mayúsculas; (b) la acción de ZAP intentó abrir un issue sin el permiso `issues: write`; (c) un job sin `checkout` heredó el `working-directory: backend` global y falló con "No such file or directory".
5. **Las fallas del entorno de desarrollo se disfrazan de bugs de la aplicación.** El registro "fallaba al azar" con `Failed to fetch` por dos causas ajenas al código de negocio: `node --watch` observaba el archivo de SQLite y reiniciaba el servidor con cada escritura, y `.env.example` traía un `CORS_ORIGINS` de ejemplo que, copiado tal cual, bloqueaba al frontend. Los ejemplos de configuración no deben traer valores que rompan el entorno local.
6. **Las advertencias de un escáner son hallazgos, no ruido.** Las dos advertencias "menores" de ZAP eran mejoras reales de seguridad (CORS abierto, respuestas cacheables).
7. **Medir antes de afirmar.** El requisito de 1 000 usuarios se daba por cubierto sin haberlo probado; medirlo mostró que el límite es el hash de contraseñas, y separar la limitación de la máquina de la de la aplicación evitó sacar una conclusión equivocada en cualquier dirección.
8. **Las dependencias de desarrollo también se auditan.** `npm audit` marcó una vulnerabilidad en una versión reciente del ejecutor de pruebas antes de que llegara a `main`; además, un fallo del resolvente de npm con las peer dependencies obligó a fijar `legacy-peer-deps` en `.npmrc` para que local y CI se comporten igual.
9. **Un detalle de pruebas puede ocultar un problema falso o uno real.** Una prueba se colgaba solo en jsdom al combinar transiciones de página con animaciones desactivadas; se comprobó en un navegador real que la aplicación no tenía el problema antes de descartarla, en vez de asumirlo.

3.4 Plan de mejora continua

Acción	Meta medible	Fecha objetivo
Hacer que el <i>quality gate</i> de SonarCloud bloquee el merge (protección de rama + análisis de PR)	0 code smells y 0 vulnerabilidades nuevas por PR; cobertura de código nuevo $\geq 80\%$	19-oct-2026
Validar el requisito de 1 000 usuarios en Linux con rampa (k6 o autocannon en CI/staging)	1 000 usuarios autenticados, p99 < 2 s, 0 errores; y login sostenido $\geq 50/s$	30-nov-2026 (Sprint 6)
Reducir el costo del login: bcrypt nativo o Argon2 en hilos + escalado horizontal	≥ 50 logins/s por instancia a costo equivalente a 12	19-oct-2026
Mover el JWT de <code>localStorage</code> a cookie <code>httpOnly</code> + <code>SameSite</code> + protección CSRF	El token no es legible desde JavaScript; pruebas de CSRF en CI	19-oct-2026
Ampliar ZAP de <i>baseline</i> (pasivo) a <i>full scan</i> autenticado, que ataca activamente los endpoints	0 alertas Alta/Media sobre <code>/api/auth/*</code> y <code>/api/vehicles</code>	10-dic-2026
Desplegar en un host persistente (Fly.io con volumen, o Render de pago con disco) y activar <code>RENDER_DEPLOY_HOOK</code>	URL de staging con uptime verificable; datos que sobreviven a un redesplicue (configuración ya lista en el repositorio)	19-oct-2026
Pruebas end-to-end del flujo registro → panel (Playwright) en CI	Flujo completo en verde en cada PR; auditoría Lighthouse de accesibilidad ≥ 95	30-nov-2026
Carga de fotos reales del vehículo (almacenamiento de objetos + validación de tipo/tamaño) y reemplazo de la ilustración	Hasta 8 fotos por auto, ≤ 5 MB c/u, con pruebas de rechazo de archivos no permitidos	Sprint 2 (28-sep → 19-oct)
Innovación: modelo para predecir la probabilidad de cierre de venta por prospecto y sugerir precio de referencia (kilometraje/año/zona), entrenado con el módulo de Reportes	Prototipo offline con AUC ≥ 0.7 sobre datos históricos simulados	Sprint 5 (16–30-nov-2026), como <i>spike</i> técnico